Yes. This branch can use the **same progressive-learning philosophy**, but instead of isolated programming questions, we'll build **one cohesive Bus Booking application** and continuously evolve it.

The key difference is that here the objective is not just:

> "Learn Angular" + "Learn Node.js" + "Learn Google Authentication"

but:

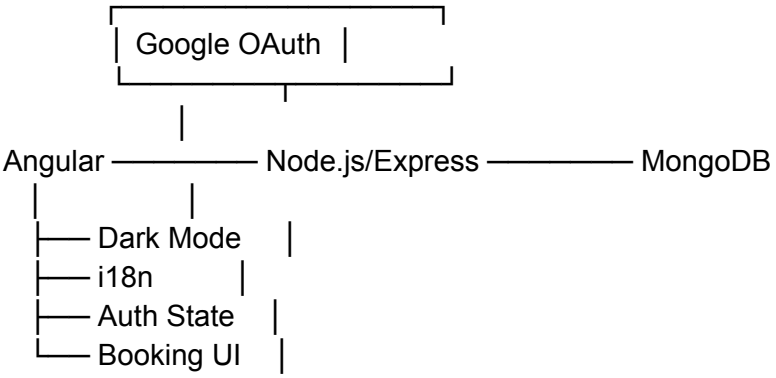
> **Build one application in which every feature forces you to understand how the frontend, backend, authentication, state, database, APIs, UI, and deployment fit together.**

Web Development Practice Track — Bus Booking System

Core stack

Frontend
Angular
  │
  │ HTTP / REST
  ▼
Backend
Node.js + Express
  │
  ▼
Database
MongoDB

With cross-cutting features:



We'll make **cohesion** a deliberate learning objective: features shouldn't exist as disconnected demos.

LEVEL W1 — Web Fundamentals

Before Angular, create the smallest possible version.

Q1 — Static Bus Search

Create a page containing:

- Source
- Destination
- Travel date
- Search button

Example:

From: Hyderabad
To: Bangalore
Date: 2026-10-15

[Search Buses]

For now, don't use Angular or Node.

Use:

HTML
CSS
JavaScript

Q2 — Display Bus Results

Hard-code:

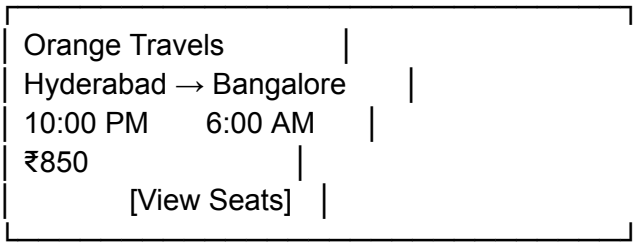
TSRTC Express
Hyderabad → Bangalore
10:00 PM
₹850

and several other buses.

Clicking **Search** should filter the displayed buses.

Q3 — Bus Card

Create a reusable conceptual bus-card structure:



Now start thinking:

> Which pieces are data and which pieces are presentation?

That distinction becomes important when Angular enters.

LEVEL W2 — Angular Foundation

Now rebuild the application in Angular.

Q4 — Angular Application

Create:

```
App
 ├── Header
 ├── Search
 ├── Bus List
 │    └── Bus Card
 └── Footer
```

Learn:

- Components
- Templates
- Interpolation
- Property binding
- Event binding
- `@Input`
- `@Output`

Q5 — Bus Model

Create a bus model/interface containing things such as:

id
operator
source
destination
departureTime
arrivalTime
price
availableSeats

Display the data through Angular.

Q6 — Angular Search Form

Build:

From
To
Date
Search

Explore Angular forms.

Eventually compare:

Template-driven forms
Reactive forms

Don't introduce backend yet.

LEVEL W3 — Angular Routing

Q7 — Application Pages

Create routes:

/
 /search
 /bus/:id
 /login
 /bookings

/profile

Navigate between them.

Q8 — Bus Details

Click:

View Seats

and navigate to:

/bus/123

Display the selected bus.

This introduces the relationship between:

route parameter
↓
component
↓
data

LEVEL W4 — Node.js Backend

Now we create the real separation.

Angular
|
HTTP
▼
Express
|
▼
Temporary in-memory data

Q9 — Create Express Server

Create:

GET /api/buses

returning JSON.

Q10 — Search API

Create:

GET /api/buses?from=Hyderabad&to=Bangalore&date=...

Angular should now retrieve the buses from Node rather than having them hard-coded.

Q11 — Bus Details API

Create:

GET /api/buses/:id

Then Angular's:

/bus/:id

page calls:

/api/buses/:id

Now we have our first real full-stack flow:

User
↓
Angular route
↓
Angular service
↓
HTTP GET
↓
Express route
↓
Controller
↓
Data
↓
JSON
↓
Angular
↓
UI

LEVEL W5 — Angular Services + Backend Architecture

Now deliberately introduce structure.

Frontend

components/
pages/
services/
models/
guards/
interceptors/

Backend

routes/
controllers/
services/
models/
middleware/
config/

Q12 — BusService

Create an Angular:

BusService

responsible for:

searchBuses()
getBus()

Components should **not directly construct HTTP requests**.

Q13 — Backend Service Layer

Separate:

route
↓
controller
↓

service

so that Express routes don't contain all business logic.

This is where we start practicing **system cohesion**, rather than simply making things work.

LEVEL W6 — MongoDB

Now replace temporary data.

Q14 — Bus Schema

Create a MongoDB model for:

Bus

with:

operator
source
destination
departure
arrival
price
totalSeats
availableSeats

Q15 — Search MongoDB

Implement:

GET /api/buses

using MongoDB queries.

Q16 — Seed Database

Create a seed mechanism that creates realistic bus data.

Now you can stop manually editing the source code whenever you want another bus.

LEVEL W7 — Seat Selection

This is where the application becomes an actual booking application.

Q17 — Seat Layout

Create:

A1 A2 B1 B2
A3 A4 B3 B4
A5 A6 B5 B6
...

with states:

available
selected
booked

Q18 — Seat Selection State

User clicks:

A3
B4

and Angular maintains:

["A3", "B4"]

Calculate:

number of seats
total price

Q19 — Booking Summary

Display:

Bus:
Hyderabad → Bangalore

Seats:
A3, B4

Price:
₹1700

[Confirm Booking]

LEVEL W8 — Google Authentication

Now authentication enters the existing system.

Not as a separate demo.

The actual application becomes:

User
↓
Google Login
↓
Authenticated Angular application
↓
Node verifies authentication
↓
User identity
↓
MongoDB user

Q20 — Login Page

Create:

Continue with Google

and a basic authenticated/unauthenticated UI state.

Q21 — User Model

Create:

User

containing appropriate identity information.

Do **not** blindly trust user information supplied by the frontend.

Q22 — Protected Backend API

Make:

/api/bookings

require authentication.

Unauthenticated request:

401 Unauthorized

Authenticated request:

continue

Q23 — Angular Auth State

Create an authentication service that understands:

anonymous
authenticated

and updates the UI accordingly.

LEVEL W9 — Actual Booking

Now combine everything we've built.

Q24 — Create Booking

Request conceptually:

POST /api/bookings

containing:

busId
travelDate

seats

The backend determines:

- authenticated user
- current seat availability
- price
- booking ID

Q25 — Prevent Double Booking

This is important.

Two users could attempt:

A3

at almost the same time.

The backend must be authoritative.

Don't solve this merely by disabling the Angular button.

LEVEL W10 — My Bookings

Q26

Create:

/my-bookings

Display:

Booking ID
Bus
Route
Date
Seats
Amount
Status

Backend:

GET /api/bookings/me

The server determines which bookings belong to the authenticated user.

LEVEL W11 — Dark Mode

Now introduce the first **cross-cutting UI feature**.

Q27 — Theme Service

Create:

ThemeService

supporting:

light
dark

The header contains:

☀ / 🌙

Q28 — Persist Theme

Reloading the page should preserve the selected theme.

Investigate appropriate browser storage mechanisms.

Q29 — Theme Every Component

Dark mode must work consistently on:

- Header
- Search
- Bus cards
- Seat map
- Booking summary
- Login
- Profile
- Booking history

- Errors
- Modals

This is where we test **system cohesion**.

A dark-mode button that only changes the homepage doesn't count.

LEVEL W12 — Internationalization / i18n

Start with:

English
Telugu
Hindi

For example:

Search Buses

becomes:

Search Buses
బస్సులను వెతకండి
बसें खोजें

Q30 — Language Selector

Add:

EN | తెలుగు | हिन्दी

to the application header.

Q31 — Translate UI

Translate:

- Search
- Login
- Logout
- Bookings
- Profile

- Select seats
- Confirm booking
- Price
- Departure
- Arrival
- Available
- Booked

Q32 — Dynamic Content

Handle:

3 seats available

properly across languages rather than simply concatenating strings.

This is where you learn why proper i18n systems use **message/translation parameters**.

LEVEL W13 — Authentication + i18n + Theme Together

Now we deliberately test feature interaction.

Q33

The user:

1. Opens application
2. Changes language to Telugu
3. Enables dark mode
4. Logs in with Google
5. Searches Hyderabad → Bangalore
6. Selects seats
7. Books
8. Opens My Bookings

Everything must remain:

Telugu
+
Dark

+
Authenticated

No feature should reset another feature.

LEVEL W14 — Error Handling

Q34 — Backend Errors

Create proper errors for:

400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict
500 Internal Server Error

Angular should display appropriate user-friendly messages.

Q35 — Network Failure

Simulate:

Backend unavailable

The UI should not simply crash.

Display something like:

Unable to connect to the server.
Please try again.

LEVEL W15 — Loading States

Every asynchronous operation should have appropriate states:

idle
loading
success

error

Apply this to:

- Search
- Bus details
- Login state
- Booking
- Booking history

LEVEL W16 — Guards & Interceptors

Q36 — Route Guard

Prevent anonymous users from accessing:

/my-bookings
/profile

Q37 — HTTP Interceptor

Centralize authentication-related HTTP behavior instead of repeating it in every service.

LEVEL W17 — Admin Side

Now expand the same application rather than creating a second project.

Add:

/admin

with:

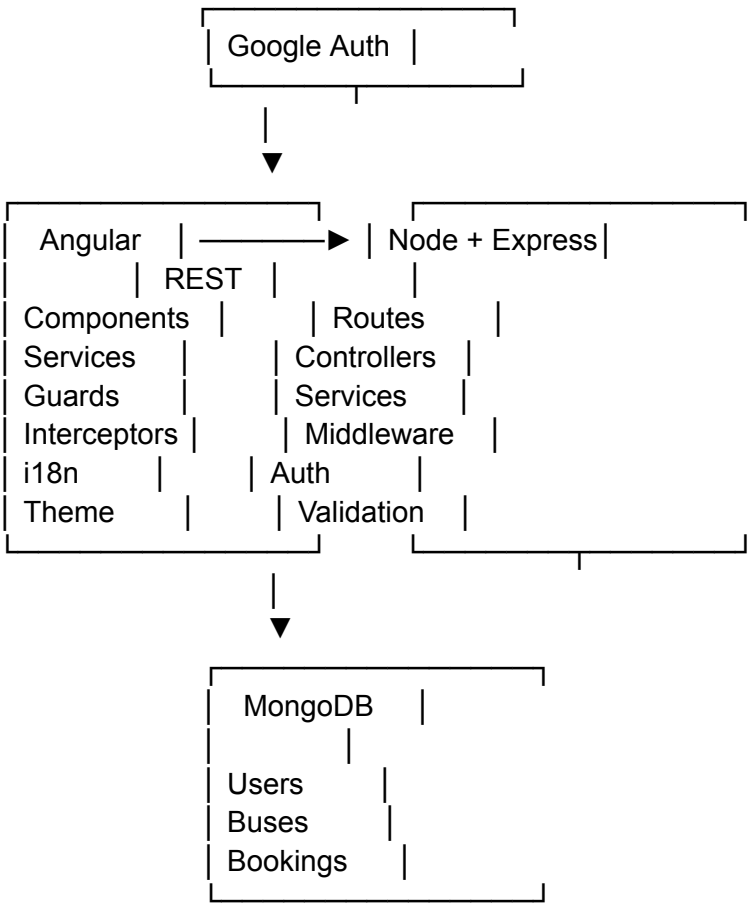
Dashboard
Buses
Bookings
Users

Admin can:

Add bus
Edit bus
Remove bus
View bookings

LEVEL W18 — Complete System

At this point your architecture should resemble:



And the **same application** demonstrates:

- Angular fundamentals
- Angular components
- routing
- forms
- services
- HTTP
- state
- guards
- interceptors
- Node.js

- Express
- REST APIs
- middleware
- authentication
- authorization
- MongoDB
- data modeling
- CRUD
- concurrency concerns
- error handling
- Google authentication
- dark mode
- internationalization
- reusable architecture
- frontend/backend separation

Most importantly: we'll make this a question-driven project

I won't simply give you a tutorial like:

> "Now create an Angular component. Here's the code."

Instead, we'll progressively give you **tasks/questions**.

For example:

> **Q17:** Create the seat-selection UI.

You attempt it.

Then:

> **Q18:** What should happen if the same seat is clicked twice?

Then:

> **Q19:** What happens if two browser tabs select the same seat?

Then:

> **Q20:** Can the frontend be trusted to say a seat is available?

Then:

> **Q21:** Where should the final validation happen?

And suddenly you've learned **Angular state + backend authority + API design + concurrency**, all through one feature.

That is the web-development equivalent of the programming-language progression we built earlier.